Error Tracing: Lessons Learned From the Trenches

Error Handling as Architecture in a Layered system

GopherCon Europe 2026

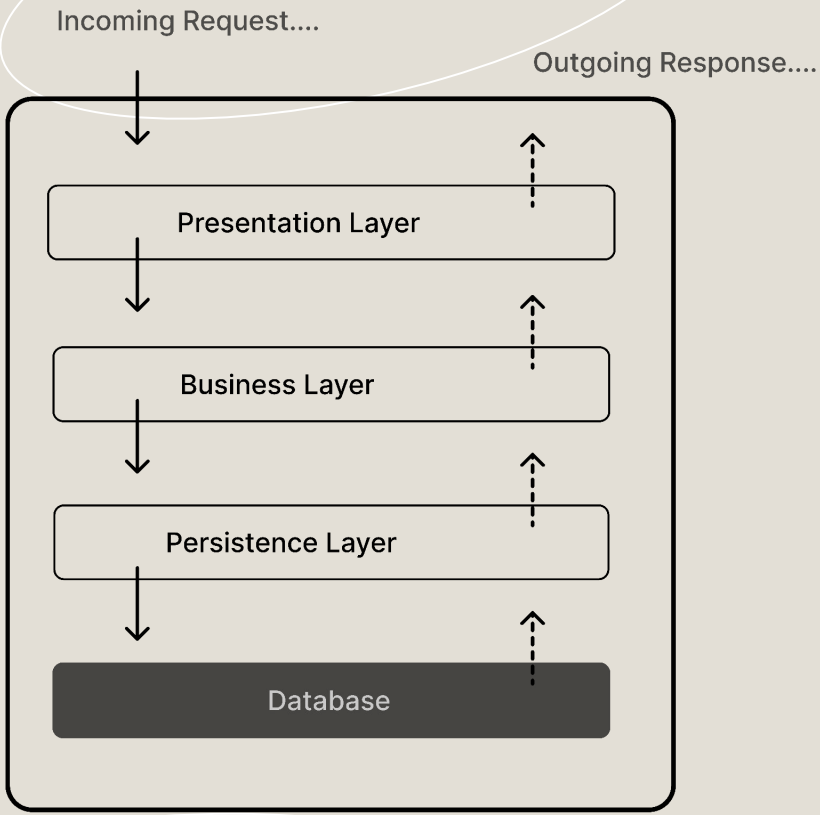
#1 The ILLusion, at least the one I tell myself

“Go Error handling is Simple”

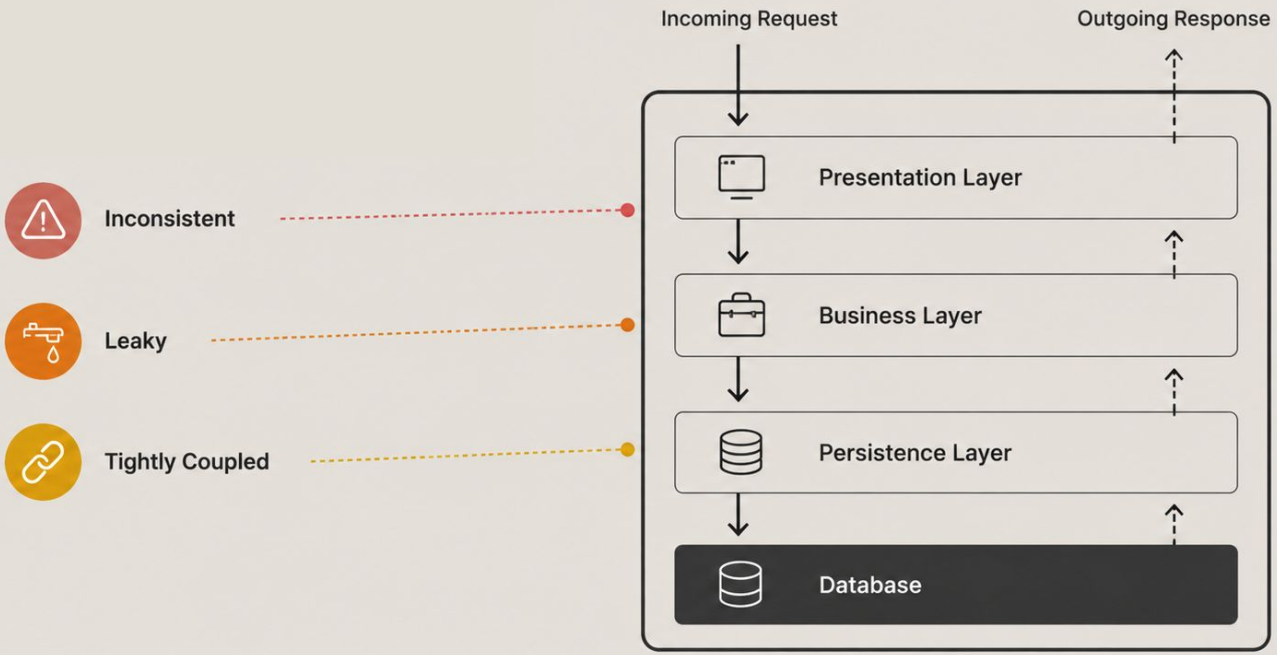
```
8  if err ≠ nil {
9      return err
10 }
```

#2

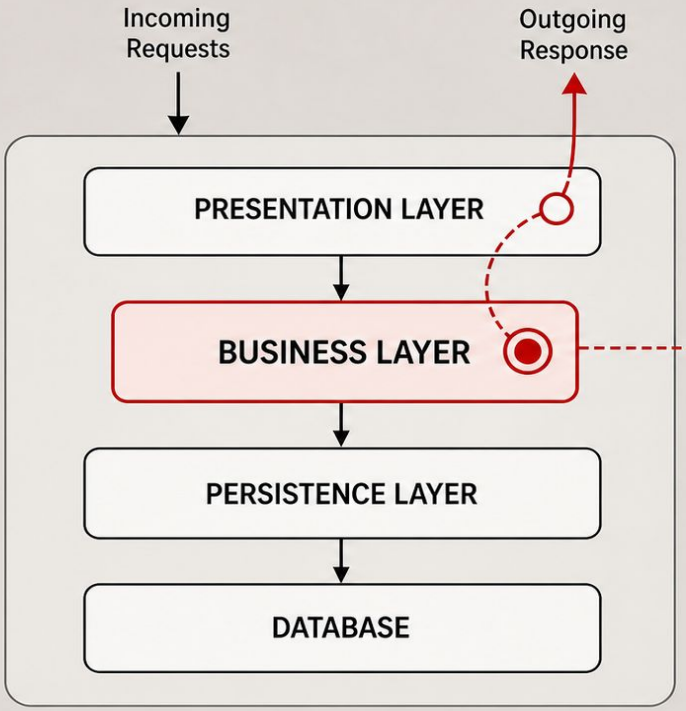
Now What Happens When Errors Starts Traveling Through Layers..?



#3 The Problem: Layered Architectures



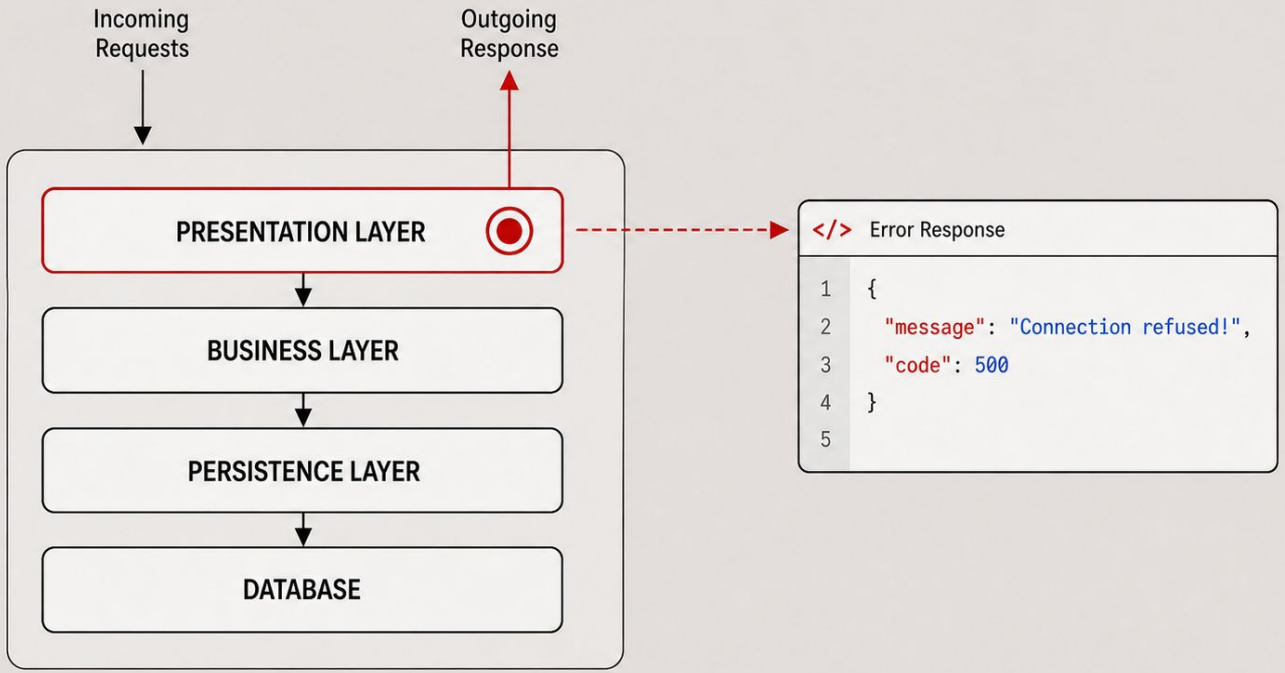
#5



</> Business Logic (Example)

```go
// creating a example order in the Business logic
func createOrder(ctx context.Context, req order) error {

    user, err := users.Get(ctx, req.UserID)
    if err /= nil {
        return err
    }

    err = inventory.Reserve(ctx, req.ItemID, req.Qty)
    if err /= nil {
        return err
    }

    err = payments.Charge(ctx, user.PaymentID, req.Total)
    if err /= nil {
        return err
    }

    return saveOrder(ctx, user.ID, req.ItemID)
}
```

#6

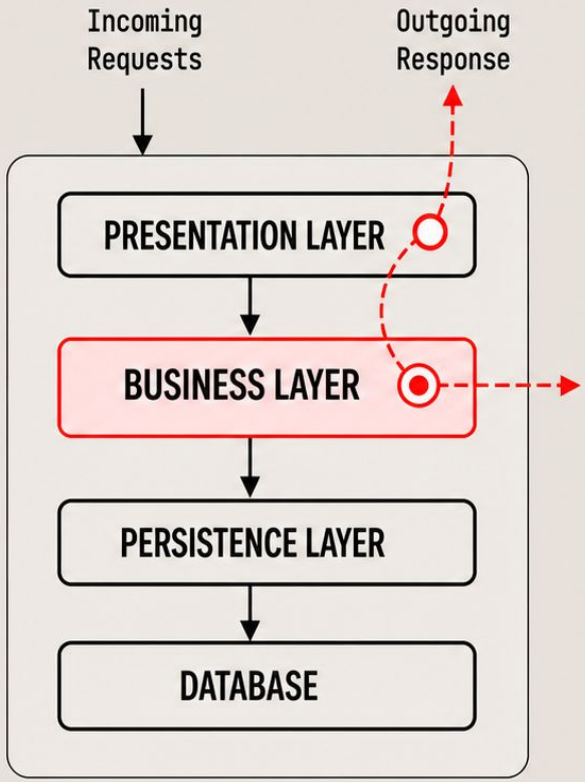

Incoming Requests
Outgoing Response
PRESENTATION LAYER
BUSINESS LAYER
PERSISTENCE LAYER
DATABASE
</> Error Response
{
"message": "Connection refused!",
"code": 500
}

#7

```go
// users persistence layer
package users

func (r *Repository) CreateUser(ctx context.Context,
    user User) (*User, error) {
    query := `sql query...`

    err := r.db.GetContext(ctx, query)
    if err != nil {
        return nil, fmt.Errorf("create user failed: %w", err)
    }
}

func (r *Repository) GetUserByID(ctx context.Context, userID string)
  (*User, error) {
    query := `SELECT * from users where id = ?`
    var user User

    err := r.db.GetContext(ctx, &user, query, userID)
    if err != nil {
        return nil, fmt.Errorf("get user by id failed: %w", err)
    }
    return &user, nil
}
```

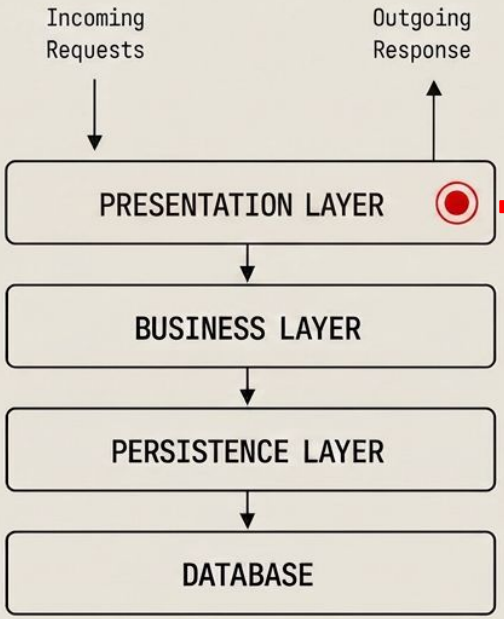
```go
// Inventory persistence layer
package Inventory

func (r *Repository) CreateProduct(ctx context.Context,
    product Product) (*Product, error) {
    query := `sql query...`

    err := r.db.GetContext(ctx, query)
    if err != nil {
        return nil, fmt.Errorf("create product failed: %w", err)
    }
}

func (r *Repository) ReserveInventory(ctx context.Context, product
    Product) (*Product, error) {
    query := `sql query...`

    err := r.db.GetContext(ctx, query)
    if err != nil {
        return nil, fmt.Errorf("reserve inventory failed: %w", err)
    }
}
```

#8




```go
// An example CreateOrder in a Buisness Layer package
func CreateOrder(ctx context.Context, req Order) error {
    user, err := users.Get(ctx, req.UserID)
    if err != nil {
        return fmt.Errorf("failed to look up user %s: %w", req.UserID, err)
    }

    err = inventory.Reserve(ctx, req.ItemID, req.Qty)
    if err != nil {
        return fmt.Errorf("failed to reserve item %s (qty: %d): %w",
                                    req.ItemID, req.Qty, err)
    }
    err = payments.Charge(ctx, user.PaymentID, req.Total)
    if err != nil {
        return fmt.Errorf("failed to charge payment %s for amount
            %.2f: %w", user.PaymentID, req.Total, err)
    }
    return saveOrder(ctx, user.ID, req.ItemID)
}

```

#9




```go
package handlers

type ErrorResponse struct {
    Message string  `json:"message"`
    Code    int     `json:"code"`
}

// defined http handler for creating order
func CreateOrderHandler(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()

    // Business layer call
    err := orders.CreateOrder(ctx, req)
    if err != nil {
        resp := ErrorResponse{
        Message:  fmt.Errorf("create order handler failed: %w", err.error()),
        Code:     http.StatusInternalServerError
    }

        writeError(w, resp)
        return
    }
    w.WriteHeader(http.StatusOk)
}
```

#10

CASE 1: USER LOOKUP ERROR


```
{
  "message": "create order
  handler failed: failed to look
  up user 123: get user by id
  failed: sql: no rows in result
  set",
  "code": 500
}
```


CASE 2: INVENTORY RESERVATION ERROR

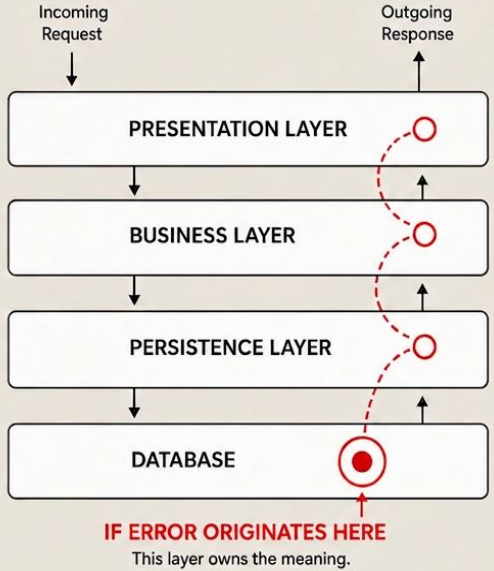

```
{
  "message": "create order
  handler failed: failed to
  reserve item item-456 (qty: 2):
  reserve inventory failed: pq:
  duplicate key value violates
  unique constraint
\\"reservations_pkey\"",
  "code": 500
}
```

#11

The Key Insight

Errors Are Not Just Return Values.

Errors should be treated as part of the Architecture.



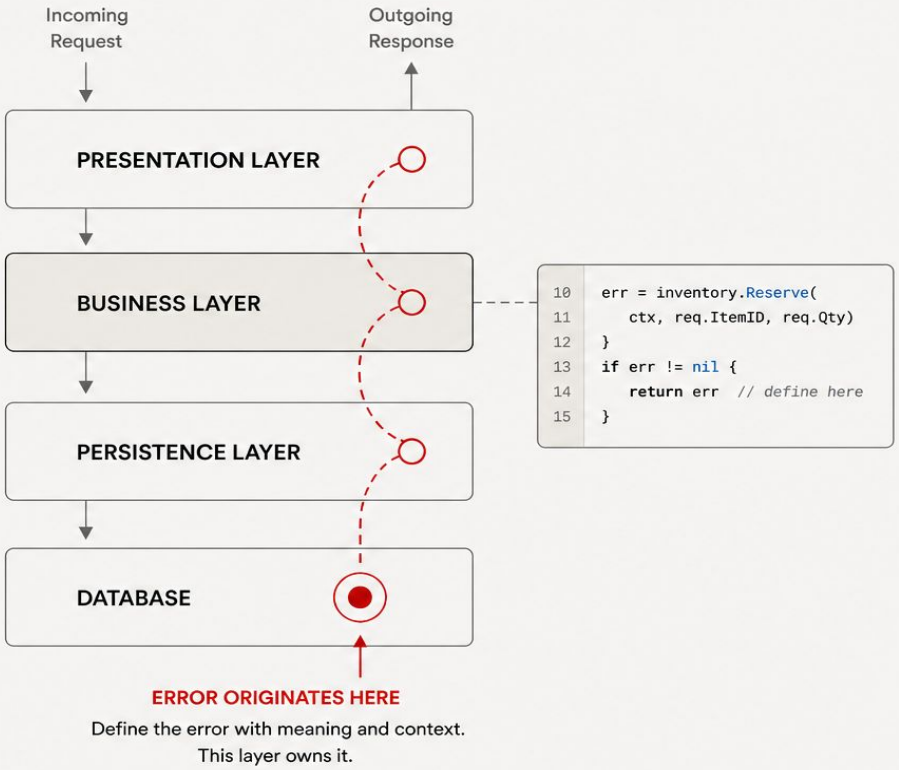
#12

THE PRINCIPLE

Define Errors at the Point of Origin

Then propagate upward unchanged.

Not every layer should reinterpret the failure.



#13

LAYERED ERROR OWNERSHIP

HTTP Should Speak HTTP

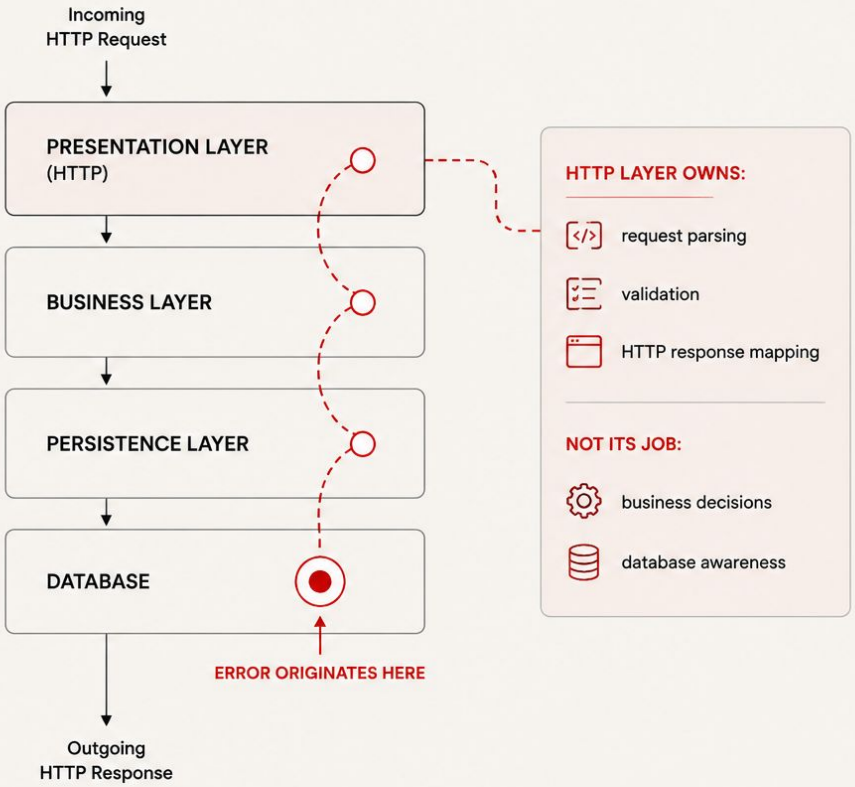
```
var ErrInvalidJSON =
  errors.New("invalid request body")
```

RESPONSIBILITIES

- request parsing
- validation
- HTTP response mapping

NOT

- business decisions
- database awareness



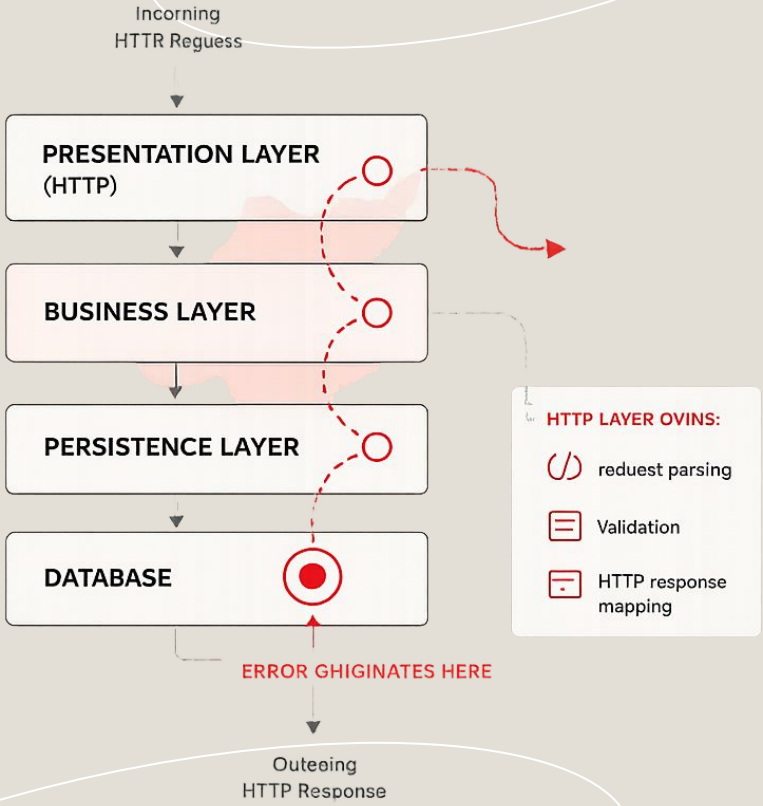
#14

BUSINESS LAYER

Business Errors Belong to the Domain

```
Var ErrActivesubstriptionExistt =
    errers New/'active subscriptton aireadv
    extsts")
```

- Express business intent
- Remain infrastructure-agnostic
- Survive across layers



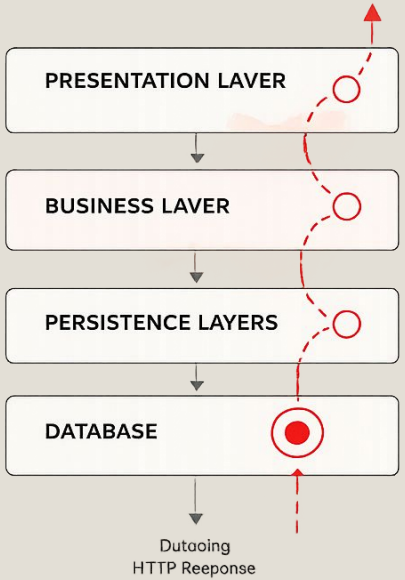
#15

THE RULE

Above the Source:

Propagate. Don't Decorate.

- If a laver did not create the meaning.
  it should not redefine the meaning.



#16

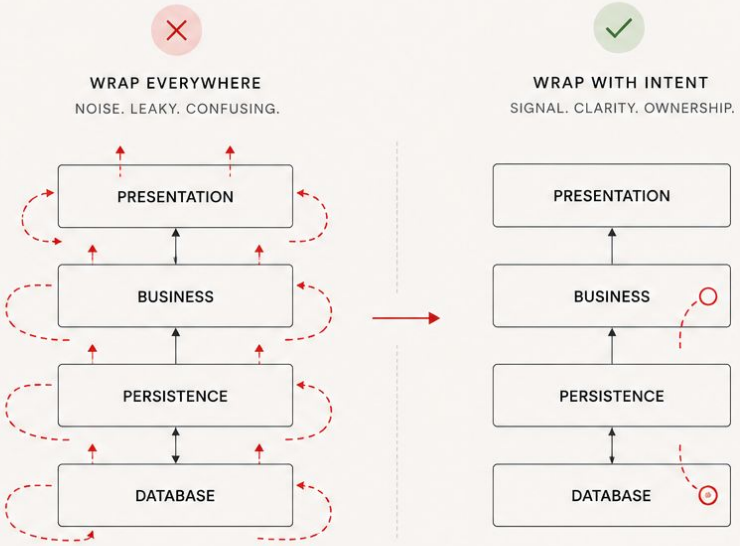
WRAPPING ISN'T EVIL

But Wrapping Everywhere Is

We are not rejecting wrapping.

We are constraining:

- where it happens
- why it happens
- who owns it



#19

Demo Part 1

The “Bad” Version

#20

Demo Part 2

The Structured Version

#21

What Changed?

Not the Errors

The architecture changed.

The system became:

- easier to reason about
- easier to debug
- easier to evolve

#22

TRADE-OFFS

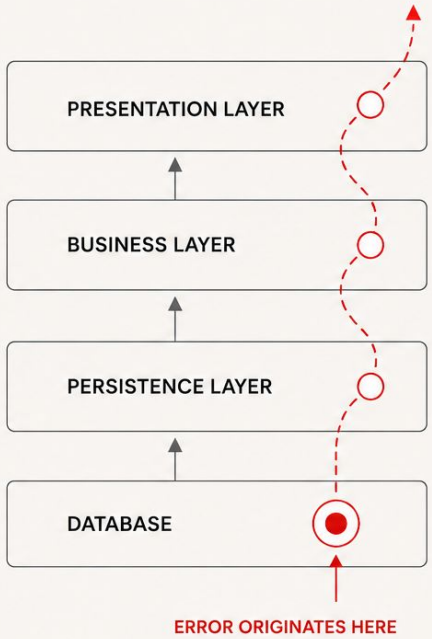
This Approach Requires Discipline

YOU MUST:

- define boundaries intentionally
- avoid unnecessary wrapping
- log correctly at the source

OTHERWISE:

- context gets lost
- ownership becomes unclear



#23

Important Clarification

This Is Not About Perfect Errors

This is about:

- preserving meaning
- reducing coupling
- enforcing architectural clarity

#24

Final Thought

Simplicity Does Not Scale Automatically

Go gives us simple primitives.

Architecture determines whether they remain simple at scale.

#24

THANK YOU!